Integración por Monte Carlo en memoria compartida con OpenMP

Integrantes:

Miguel Angel Parada Rodríguez - 2201167

Sebastián Villamizar Saavedra - 2230078

David Steven Anaya González - 2211714

Sergio Andrés Pinzón Castellanos - 2202006

Nancy Liliana Saenz Moreno - 2224510

Introducción a la Programación Paralela

Universidad Industrial de Santander

2026-2

Parte I. Fundamento del método

La integración por Monte Carlo en pocas palabras

La integración por Monte Carlo estima el valor de una integral mediante muestreo aleatorio. Para aproximar

$$I = \int_a^b f(x)\,dx,$$

se generan N puntos $x_1, x_2, \ldots, x_N$ distribuidos de manera uniforme en $[a, b]$ y se calcula el estimador

$$I_N = \frac{b-a}{N} \sum_{i=1}^{N} f(x_i).$$

Por la ley de los grandes números, I_N converge al valor exacto de la integral cuando N crece; el error estadístico típico disminuye en proporción a $1/\sqrt{N}$, de modo que cuadruplicar el número de muestras reduce el error a la mitad.

Un caso particular le servirá en esta evaluación: el área del cuarto de círculo unitario es $\pi/4$, es decir,

$$\int_0^1 \sqrt{1-x^2}\,dx = \frac{\pi}{4}.$$

En la variante de *acierto y fallo*, se generan puntos (x, y) uniformes en el cuadrado $[0, 1] \times [0, 1]$ y se cuenta la fracción que cae dentro del cuarto de círculo $x^2 + y^2 \leq 1$; esa fracción multiplicada por 4 aproxima π. Cada muestra es independiente de las demás, y lo único que se acumula a lo largo del cálculo es el número de aciertos.

Preguntas

I.1. Explique con sus propias palabras por qué el error del estimador disminuye en proporción a 1/ √ N y deduzca cuántas muestras adicionales se requieren para reducir el error a la mitad. ¿Qué implicación tiene esto sobre el costo de ganar un dígito de precisión?

Para hacer explicación del error del estimador, es necesario recapitular la información proporcionada anteriormente. Teniendo en cuenta lo ya mencionado, se ha optado por usar la variante de acierto y fallo que se explicó en "*La integración por Monte Carlo en pocas palabras*".

Cito: “En la variante de acierto y fallo, se generan puntos (x,y) uniformes en el cuadrado [0,1] x [0,1] y se cuenta la fracción que cae dentro del cuarto de círculo $x^2+y^2 <= 1$; esta fracción multiplicada por 4 aproxima π .” Ok, entonces tenemos lo siguiente:

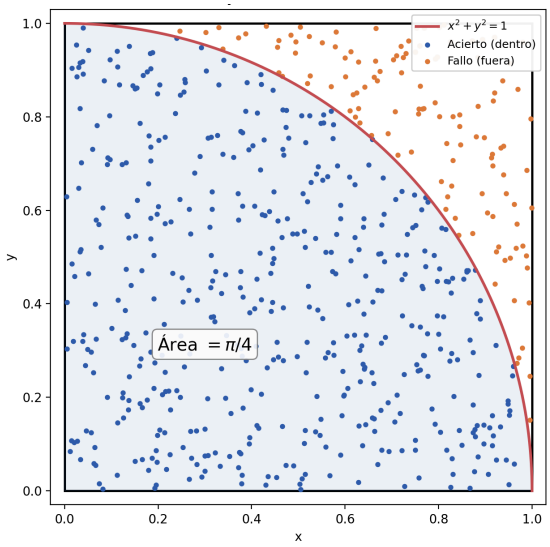


Figura 1. Imagen representativa de lo planteado por la variante acierto y fallo

A sabiendas que el área del cuarto del círculo es π/4 (véase el resultado de la integral) y la de un cuadrado está dada por L^2, siendo su valor igual a 1. Si se tiene en cuenta que los puntos están distribuidos de manera uniforme en el cuadrado, entonces la proporción de puntos que cae dentro del cuarto de círculo tiende a ser igual a la proporción del área que ocupa.

Por tanto:

$$\frac{Área\ del\ Cuarto\ de\ Círculo}{Área\ del\ cuadrado} \approx \frac{Puntos\ dentro\ del\ círculo}{Puntos\ totales}$$

Sustituimos valores y simplificamos

$$\frac{\pi}{4} \approx \frac{Puntos\ dentro\ del\ círculo}{Puntos\ totales}$$

Para eliminar la fracción y dejar el aproximado de π , entonces multiplicamos por 4 en ambos lados:

$$\pi \approx 4\frac{Puntos\ dentro\ del\ círculo\ (M)}{Puntos\ totales\ (N)}$$

Ahora, tomemos que la proporción de aciertos como un estimador probabilístico, $\hat{p} = \frac{M}{N}$. Dónde, si tenemos en cuenta el proceso anterior tenemos que $\frac{\pi}{4} \approx \hat{p}$. Pero, ahora me gustaría saber qué tanto se equivoca esta estimación. Para eso modelo cada punto como un ensayo de Bernoulli independiente, como una moneda que cae “acierto” o “fallo” :

$$X_i = \begin{cases} 1, & \text{si el punto } (x_i, y_i) \text{ cae dentro del círculo } (x_i^2 + y_i^2 \leq 1) \\ 0, & \text{en caso contrario} \end{cases}$$

Como los puntos son uniformes en un cuadrado de área 1, la probabilidad de acierto es el área del cuarto de círculo. Aquí llamo p a esa probabilidad teórica, que no es lo mismo que p̂, la proporción que se observa en una corrida:

$$P(X_i = 1) = p = \frac{\pi}{4}$$

Al ser Bernoulli, cada X_i tiene esperanza y varianza conocidas:

- $E[X_i] = p = \frac{\pi}{4}$
- $Var(X_i) = \sigma^2 = p(1 - p) = \frac{\pi}{4}\left(1 - \frac{\pi}{4}\right)$

Mi proporción observada es el promedio de los X_i: $\hat{p} = \frac{1}{N}\sum_{i=1}^{N} X_i$. Como cada punto se lanza de forma independiente de los demás, la varianza de la suma es la suma de las varianzas:

$$\mathrm{Var}(\hat{p}) = \mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} X_i\right) = \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(X_i) = \frac{N \;\; p(1-p)}{N^2} = \frac{p(1-p)}{N}$$

Dado que la estimación final de π es $\hat{\pi} = 4\hat{p}$, su varianza y su desviación estándar (error típico estadístico $Error(N)$) son:

$$Var(\hat{\pi}) = 16 \cdot Var(\hat{p}) = \frac{16p(1-p)}{N}$$

$$Error(N) = \sqrt{Var(\hat{\pi})} = \frac{4\sqrt{p(1-p)}}{\sqrt{N}} \propto \frac{1}{\sqrt{N}}$$

Ahora, queremos determinar un nuevo valor de N que permita reducir el valor del error a la mitad. Entonces, sencillamente hacemos este proceso:

$$Error(N(new) = \frac{1}{2\sqrt{N}}$$

$$\frac{1}{\sqrt{N(new)}} = \frac{1}{2\sqrt{N}}$$

Despejamos el valor N(new)

$$\sqrt{N(new)} = 2\sqrt{N}$$

$$(\sqrt{N(new)})^2 = (2\sqrt{N})^2$$

$$N(new) = 4N$$

Teniendo en cuenta el dato anterior, sencillamente se hace un diferencia entre el valor nuevo de N con el valor anterior para obtener las muestras adicionales necesarias.

$$\Delta N = 4N - N$$

$$\Delta N = 3N$$

Por lo tanto **la cantidad de muestras adicionales para reducir el error a la mitad es de 3N**, dónde N es la cantidad de muestras totales.

Pero si se busca ganar **un dígito de precisión decimal** en la estimación de π equivale a reducir el error estadístico por un factor de 10 (es decir, conseguir un error de $\frac{1}{10}Error(N)$), seguimos la misma lógica anterior:

$$\frac{1}{\sqrt{N_{\text{nuevo}}}} = \frac{1}{10} \cdot \frac{1}{\sqrt{N}} \implies \sqrt{N_{\text{nuevo}}} = 10\sqrt{N} \implies N_{\text{nuevo}} = 100N$$

Entiéndase que el costo computacional en Monte Carlo escala de forma cuadrática con respecto a la precisión obtenida. Teniendo en cuenta el proceso anterior, para reducir un dígito, se requiere aumentar el volumen de cómputo por un factor de 100. Por lo tanto, el método resultaría útil para obtener aproximaciones rápidas ya sea de baja o mediana precisión, pero se vuelve sumamente costoso cuando se exige precisiones numéricas muy elevadas de forma secuencial.

I.2. Justifique por qué la integración por Monte Carlo se clasifica como un problema embarazosamente paralelo: analice las dependencias entre las muestras (o su ausencia) y determine qué datos deben ser compartidos y cuáles deben ser privados cuando el muestreo se reparte entre varios hilos de ejecución.

La integración por Monte Carlo resulta ser un problema embarazosamente paralelo porque el proceso principal consiste en generar puntos aleatorios e independientes para evaluar condiciones geométricas sin que exista cómo tal una dependencia o comunicación intermedia entre las muestras. Véase en la variante de acierto y fallo, dónde se generan puntos uniformes (x,y) en el cuadrado [0,1] x [0,1] para contar cuántos caen dentro del cuarto de círculo definido por $x^2+y^2 \leq 1$. Si lo miras bien, cada muestra es

completamente independiente de las demás y lo único que cómo tal se acumula a lo largo de todo el cálculo es el número de aciertos, por lo que el problema puede dividirse en múltiples hilos de ejecución sin necesidad de sincronización en la fase de muestreo.

Pero, al repartir este proceso en un entorno de memoria compartida utilizando OpenMP, es necesario definir con precisión el ámbito de cada variable para evitar las condiciones de carrera y garantizar la reproducibilidad. Por el lado de las coordenadas temporales de los puntos (x,y), los índices de iteración y el estado interno del generador de números pseudoaleatorios deben configurarse como datos privados por hilo. Esto resulta indispensable para que se asegure que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento, utilizando funciones como `rand_r(&estado)` con semillas espaciadas adecuadamente a partir de la semilla base asignada.

Por otro lado, los parámetros de lectura global como el número total de muestras (N) son compartidos, mientras que el acumulador global de aciertos se gestiona mediante una cláusula de reducción adecuada (***reduction*** de OpenMP). Esta cláusula permite que cada hilo acumule sus aciertos en contadores locales independientes durante el ciclo paralelo, combinándolos de forma segura y atómica al finalizar la ejecución para consolidar el resultado total sin corromper los datos.

I.3. Demuestre que repartir las N muestras entre p hilos y sumar los aciertos parciales de cada uno produce el mismo estimador que la versión secuencial. Con el mismo N total, ¿qué precisión estadística cabe esperar respecto de la ejecución con un solo hilo? Justifique.

Para demostrar que ambos enfoques producen el mismo estimador, es necesario tener en cuenta de cómo se calcula la estimación de π en la variante de acierto y fallo. Por el lado de la versión secuencial se procesan N puntos, y a cada punto m se le asigna un valor que es 1 si cae dentro del cuarto de círculo ($x_m^2 + y_m^2 \leq 1$) y 0 en caso contrario. El número total de aciertos es $C_{secuencial} = \sum_{m=1}^{N} c_m$, y la estimación de π es:

$$I_{N,secuencial} = \frac{4}{N} \sum_{m=1}^{N} c_m$$

Para la versión paralela, las N muestras se reparten entre p hilos. Cada hilo k procesa un bloque de N_k muestras y calcula su número de aciertos parcial $C_k = \sum_{i=1}^{N_k} c_{k,i}$. Los bloques son disjuntos, porque ninguna muestra la procesan dos hilos, y su unión es el conjunto completo, porque ninguna queda sin procesar. Por eso $\sum_{k=1}^{p} N_k = N$ y cada par (k, i) corresponde exactamente a una muestra m. Al terminar, la reducción suma los parciales:

$$C_{paralelo} = \sum_{k=1}^{p} C_k$$

Como cada muestra se cuenta una sola vez y la suma de enteros es asociativa y conmutativa, reagrupar los términos en sumas parciales no cambia el total. Por tanto $C_{paralelo} = C_{secuencial}$ y, al evaluar las mismas N muestras, se obtiene $I_{N,paralelo} = I_{N,secuencial}$. Repartir el trabajo entre varios hilos cambia la forma de ejecutarlo, pero no el resultado del estimador. Esto vale cuando ambos programas usan los mismos puntos. Si cada hilo usara su propio flujo aleatorio, los puntos serían distintos y los resultados no coincidieron dígito a dígito, aunque los estimadores seguirán siendo equivalentes en distribución, con la misma esperanza y la misma varianza.

En cuanto a la precisión estadística, con el mismo número total de muestras N cabe esperar la misma precisión que con un solo hilo. El error estándar del estimador disminuye como $1/\sqrt{N}$, según lo visto en I.1, y en esa expresión solo interviene el número total de muestras independientes, no la cantidad de hilos. Repartir el trabajo no cambia cuántas muestras hay ni su independencia. En consecuencia, aumentar el número de hilos reduce el tiempo de ejecución, pero no mejora la precisión mientras N permanezca constante. Esta conclusión supone que los flujos de números aleatorios de los hilos son independientes y no se solapan; si estuvieran correlacionados, las muestras efectivas serían menos de N y el error real sería mayor.

Parte II. Implementación en OpenMP

El siguiente programa secuencial servirá de punto de partida. Estúdielo antes de modificarlo: su estructura es la que usted debe conservar en la versión paralela.

```
/* montecarlo.c -- version secuencial de referencia.
   Uso: ./montecarlo <numero de muestras> */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
     long N = (argc > 1) ? atol(argv[1]) : 100000000L;
     unsigned int estado = 20261018u; /* semilla del generador */
     long dentro = 0;

     for (long i = 0; i < N; i++) {
     double x = rand_r(&estado) / (double)RAND_MAX;
     double y = rand_r(&estado) / (double)RAND_MAX;
     if (x * x + y * y <= 1.0)
            dentro++;
     }

     printf("Estimacion de pi: %.6f\n", 4.0 * dentro / (double)N);
     return 0;
}
```

Compílelo con: gcc -O2 montecarlo.c -o montecarlo -lm. Cuando avance a la versión paralela, agregue la bandera -fopenmp; esta también le permitirá medir tiempos con omp_get_wtime(), que necesitará desde la parte II.

Tareas

Todo lo ejecutamos en nuestro portátil (4 núcleos, 8 hilos), con $N = 10^8$ puntos y la semilla 20261018.

II.1 Compile y ejecute la versión secuencial con $N = 10^8$. Verifique que la estimación es consistente con el valor de π (un error del orden de 10^{-4} es normal; explique de dónde sale ese orden de magnitud). Registre el tiempo de ejecución T(1) con omp_get_wtime() y consérvelo para la parte III.

Corrimos el programa tal como viene y dio **$\pi \approx$ 3,141633**. El valor real es 3,141593, así que solo nos equivocamos por 0,00004.

¿Por qué es normal un error de alrededor de 0,0001? Porque el resultado depende del azar, y el error baja como $1/\sqrt{N}$. Con 100 millones de puntos, $\sqrt{N} = 10\,000$, entonces el error esperado ronda $1/10\,000 = 0{,}0001$ (la fórmula exacta da 0,000164). Nuestro error quedó por debajo de eso, así que el resultado es correcto.

Tiempo: lo medimos con omp_get_wtime() en 7 corridas y tomamos el valor del medio (la mediana): **T(1) = 1,3570 s**. Este es el valor que usa la parte III.

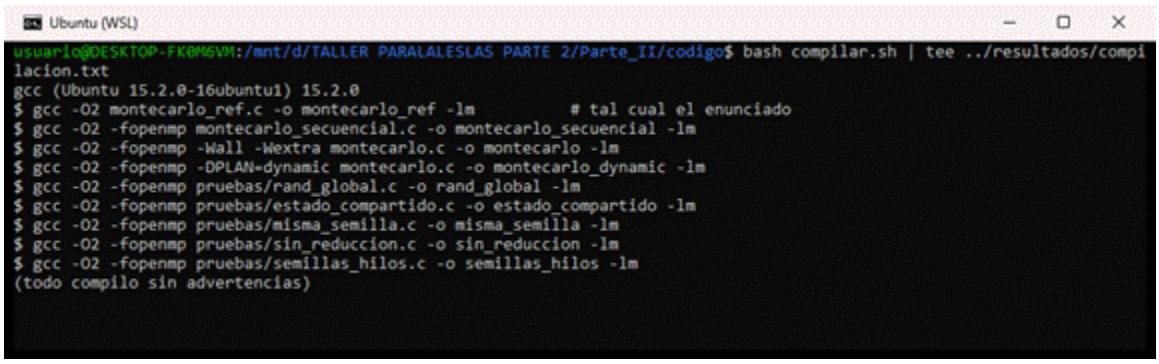


Figura II-1. Todo compila sin errores ni advertencias.

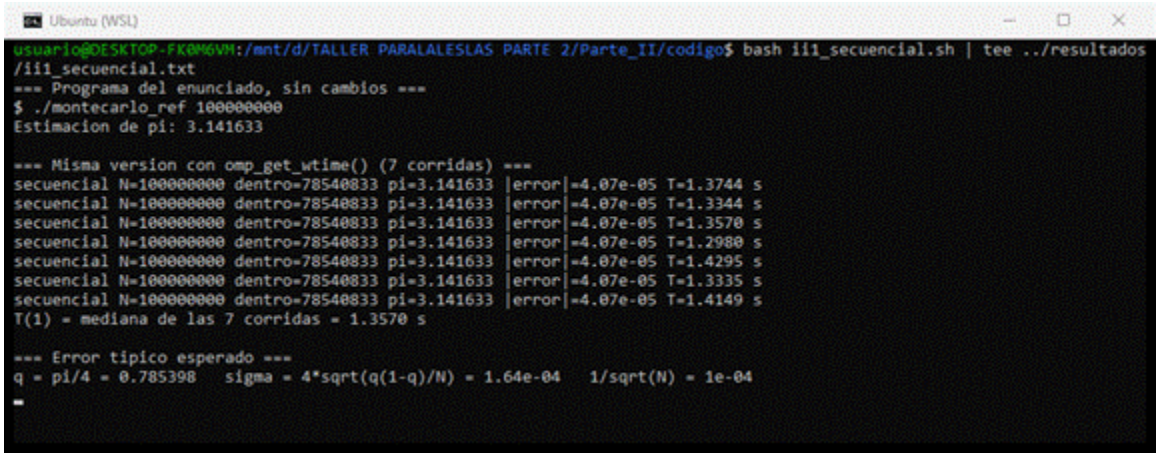


Figura II-2. II.1: π = 3,141633 y T(1) = 1,3570 s (mediana de 7 corridas).

II.2 Implemente la versión paralela con OpenMP. Su programa debe:

- **usar una directiva omp parallel for con la cláusula de reducción adecuada sobre el acumulador de aciertos;**
- **permitir elegir el número de hilos p (con omp_set_num_threads o OMP_NUM_THREADS);**
- **garantizar que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento con el de los demás. Como semilla base use la que su docente le asignó (las semillas se publican por estudiante) y espacie las de cada hilo al menos N/p valores. El resultado debe ser reproducible: con las mismas semillas, el programa debe producir la misma estimación cualquiera que sea el valor de p.**

La idea fue no cambiar el programa original, solo repartir el trabajo. Dividimos los 100 millones de puntos en 1000 paquetes de 100 000, y OpenMP reparte los paquetes entre los hilos (el código completo está en el Anexo II-A):

```
#pragma omp parallel for reduction(+:dentro) schedule(static)
for (long b = 0; b < nbloques; b++) {
    unsigned int estado = saltar(semilla, b * BLOQUE * 6);
    /* aqui va el mismo ciclo del programa original */
}
```

Así cumplimos cada punto:

- **Reducción:** con reduction(+:dentro) cada hilo cuenta sus aciertos por su lado y al final se suman.
- **Elegir los hilos:** se indica al ejecutar (./montecarlo 100000000 4) o con OMP_NUM_THREADS.
- **Números que no se repiten:** los números aleatorios salen de una lista fija que empieza en la semilla. La función saltar lleva cada paquete directo a su parte de la lista. Es como repartir un libro por capítulos: cada hilo lee páginas distintas.

Con 4 hilos, cada uno empieza 25 millones de puntos (N/p) después del anterior, que es el espaciado que pide el enunciado (Figura II-3).

- **Siempre el mismo resultado:** cada paquete usa siempre sus mismos números, así que no importa qué hilo lo haga. Con 1, 2, 4 y 8 hilos obtuvimos siempre **78 540 833 aciertos**, igual que el programa original. Y con 8 hilos el tiempo bajó de 1,56 s a 0,30 s.

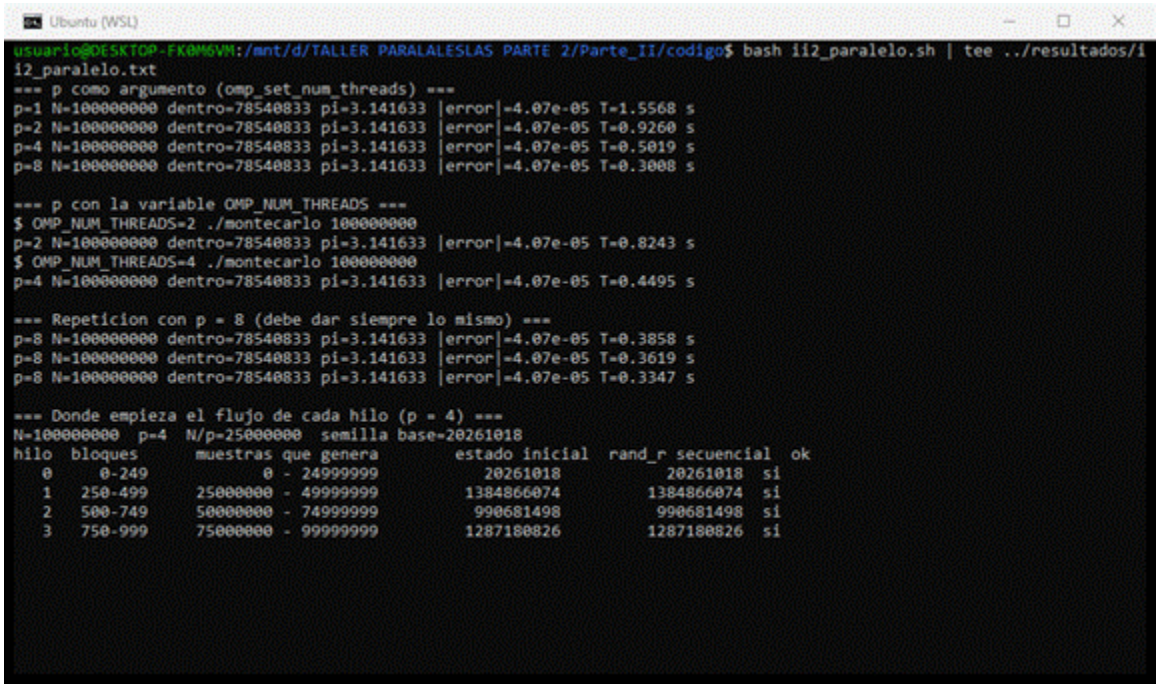


Figura II-3. II.2: el mismo resultado con 1, 2, 4 y 8 hilos; abajo, dónde empieza cada hilo con p = 4.

II.3 Explique por qué no es correcto que todos los hilos llamen a la función rand() compartiendo el estado global del generador. Su respuesta debe cubrir al menos dos problemas distintos: uno de condición de carrera sobre el estado interno, y otro de correlación entre secuencias si cada hilo inicializara el generador con la misma semilla. Indique qué consecuencias tendría cada uno sobre la estimación de π y sobre el error reportado.

rand() guarda su posición en un único lugar que comparten todos los hilos. Eso causa dos problemas, y los probamos con programas hechos a propósito (Figura II-4).

1. Condición de carrera. Es como 8 personas escribiendo en el mismo cuaderno al mismo tiempo: se pisan. Dos hilos pueden tomar el mismo número o saltarse otros.

- **En π:** el resultado cambia en cada ejecución (78 540 943, 78 539 115 y 78 540 976 aciertos). Además es más lento: con 8 hilos tardó 15 s, contra 2 s con uno, porque los hilos tienen que turnarse.
- **En el error:** como se repiten números, en realidad hay menos puntos de los que creemos. El error real es mayor que el que reportamos.

2. La misma semilla en todos los hilos. Si cada hilo tiene su propio generador pero todos empiezan igual, todos lanzan exactamente los mismos puntos.

- **En π:** con 8 hilos dio 3,142494, idéntico a usar solo 12,5 millones de puntos (N/8). Copiar los mismos puntos 8 veces no ayuda en nada.
- **En el error:** creemos tener la precisión de 100 millones de puntos (±0,00016), pero el error real fue de 0,0009, más de 5 veces peor.

Nuestra versión evita los dos problemas: cada hilo tiene su propio generador y cada paquete empieza en un lugar distinto de la lista.

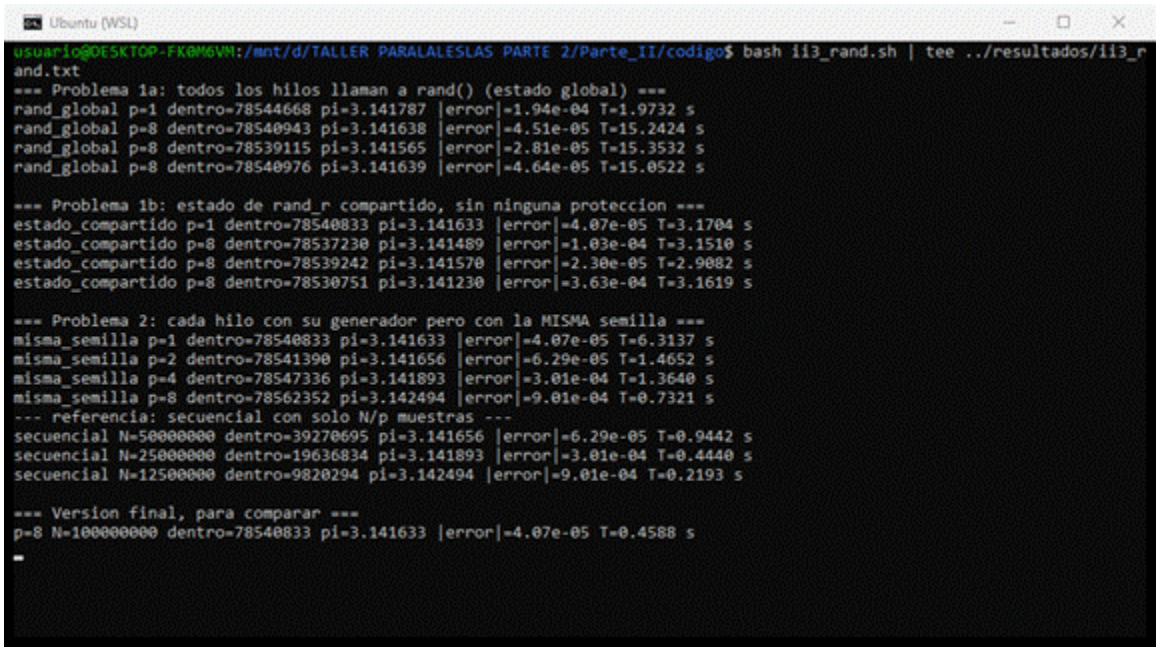


Figura II-4. II.3: con rand() compartido, el resultado cambia y es más lento; con la misma semilla, 8 hilos dan lo mismo que N/8 puntos.

II.4 Analice los atributos de compartición de datos de su región paralela: enumere las variables que OpenMP trata como compartidas y como privadas por defecto en su código, explique la cláusula que usó para el acumulador y describa qué ocurriría si el acumulador se actualizara dentro del ciclo sin esa cláusula (diagnostique el tipo de defecto y su efecto sobre el resultado).

Una variable **compartida** es la misma para todos los hilos; una **privada** es una copia propia de cada hilo. Para no adivinar, le pedimos al compilador que nos dijera cuáles eran compartidas (con default(none)), y nombró N, nbloques y semilla (Figura II-5).

Variable	Tipo	Por qué
N, nbloques, semilla	compartidas	se crean antes del ciclo y solo se leen
b	privada	es el contador del ciclo paralelo

estado, fin, i, x, y	privadas	se crean dentro del ciclo
dentro	privada gracias a reduction	es el contador de aciertos

Que N, nbloques y semilla sean compartidas no causa problemas, porque nadie las modifica.

Cláusula usada: reduction(+:dentro). Cada hilo cuenta sus aciertos en su propia "libreta", que empieza en 0, y al final OpenMP suma todas las libretas.

Sin esa cláusula: todos sumarían en el mismo contador a la vez. Sumar 1 son tres pasos (leer, sumar y guardar), y si dos hilos lo hacen al mismo tiempo, uno borra la suma del otro. Es una **condición de carrera**, y se pierden aciertos. Lo probamos:

- con 1 hilo da bien (78 540 833), por eso el error no se nota si solo se prueba sin paralelismo;
- con 8 hilos π salió **0,81; 0,60 y 0,63**: se perdió entre el 74 % y el 81 % de los aciertos, y cambió en cada ejecución.

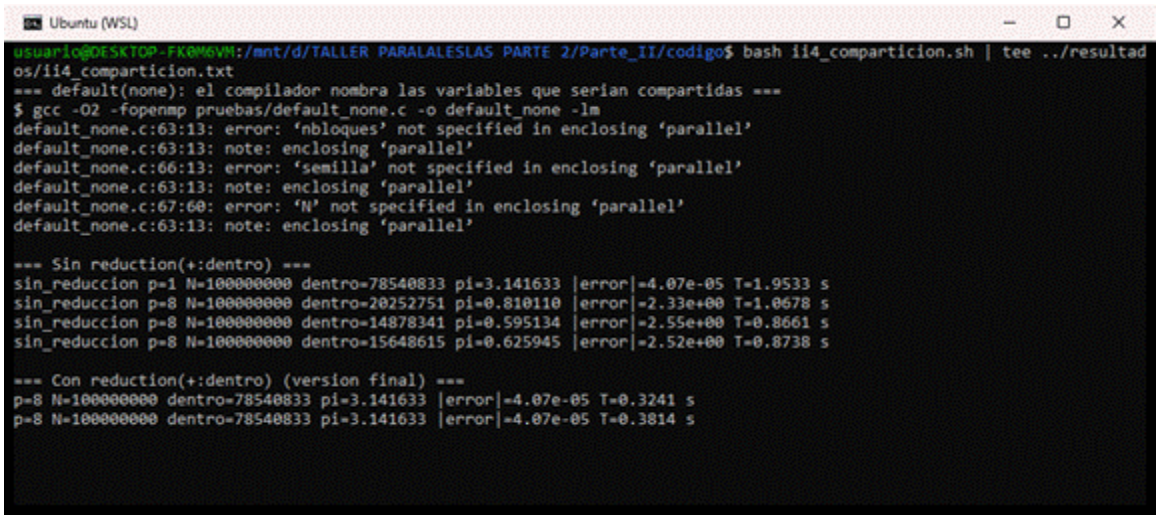


Figura II-5. II.4: el compilador nombra las variables compartidas; sin reduction π sale mal (0,60 a 0,81); con reduction sale bien.

Anexo II-A. Programa paralelo completo (montecarlo.c)

```
/* montecarlo.c -- version paralela con OpenMP (version final).
   Uso: ./montecarlo <numero de muestras> [hilos] [semilla]
     (el numero de hilos tambien puede fijarse con OMP_NUM_THREADS)
   Compilar: gcc -O2 -fopenmp montecarlo.c -o montecarlo -lm

   Idea: la secuencia de rand_r con la semilla base se parte en bloques
   consecutivos de BLOQUE muestras. El estado inicial de cada bloque se
```

```
   calcula "saltando" hacia adelante en el generador, asi que cada bloque
   consume siempre los mismos numeros, sin importar que hilo lo ejecute.
   Resultado: flujos disjuntos (sin solapamiento) y la misma estimacion
   para cualquier p, identica ademas a la de la version secuencial. */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define SEMILLA_BASE 20261018u /* reemplazar por la semilla asignada */
#define BLOQUE 100000L         /* muestras por bloque */

#ifndef PLAN
#define PLAN static            /* -DPLAN=dynamic para otra planificacion */
#endif

/* rand_r de glibc avanza su estado con 3 pasos del LCG
   s <- 1103515245*s + 12345 (mod 2^32) en cada llamada,
   y cada muestra (x, y) hace 2 llamadas: 6 pasos por muestra. */
#define LCG_A 1103515245u
#define LCG_C 12345u
#define PASOS_POR_MUESTRA 6ULL

/* Avanza el estado s en k pasos del LCG con O(log k) operaciones. */
static unsigned int saltar(unsigned int s, unsigned long long k) {
    unsigned int a = LCG_A, c = LCG_C;  /* transformacion de 2^j pasos */
    unsigned int A = 1u, C = 0u;      /* transformacion acumulada */
    while (k) {
    if (k & 1ULL) {
          A = a * A;
          C = a * C + c;
    }
    c = a * c + c;
    a = a * a;
    k >>= 1;
    }
    return A * s + C;
}

int main(int argc, char *argv[]) {
    long N = (argc > 1) ? atol(argv[1]) : 100000000L;
    if (argc > 2) omp_set_num_threads(atoi(argv[2]));
    unsigned int semilla = (argc > 3) ? (unsigned int)strtoul(argv[3], NULL,
10)
                                      : SEMILLA_BASE;

    /* El salto debe coincidir con lo que hace rand_r en esta libc. */
    unsigned int prueba = semilla;
    rand_r(&prueba);
    rand_r(&prueba);
    if (prueba != saltar(semilla, PASOS_POR_MUESTRA)) {
    fprintf(stderr, "error: rand_r de esta libc no usa el LCG esperado\n");
    return 1;
    }
    /* Periodo del estado: 2^32. Con mas de 2^32/6 muestras la secuencia
```

```
    se repetiria y los bloques dejarian de ser disjuntos. */
    if ((unsigned long long)N * PASOS_POR_MUESTRA > 4294967296ULL)
    fprintf(stderr, "aviso: N supera el periodo del generador\n");

    long nbloques = (N + BLOQUE - 1) / BLOQUE;
    long dentro = 0;

    double t0 = omp_get_wtime();
    #pragma omp parallel for reduction(+:dentro) schedule(PLAN)
    for (long b = 0; b < nbloques; b++) {
    unsigned int estado =
          saltar(semilla, (unsigned long long)b * BLOQUE * PASOS_POR_MUESTRA);
    long fin = (b + 1) * BLOQUE < N ? (b + 1) * BLOQUE : N;
    for (long i = b * BLOQUE; i < fin; i++) {
          double x = rand_r(&estado) / (double)RAND_MAX;
          double y = rand_r(&estado) / (double)RAND_MAX;
          if (x * x + y * y <= 1.0)
                dentro++;
    }
    }
    double t = omp_get_wtime() - t0;

    double pi = 4.0 * dentro / (double)N;
    printf("p=%d N=%ld dentro=%ld pi=%.6f |error|=%.2e T=%.4f s\n",
          omp_get_max_threads(), N, dentro, pi, fabs(pi - M_PI), t);
    return 0;
}
```

Anexo II-B. Bitácora de IA (Parte II)

#	Herramienta	Propósito	Qué aportó	Qué se aceptó / descartó y por qué
6	Claude (Claude Code, modelo Opus 5.5), 26/10/2026	Apoyo para responder II.1 a II.4	Los programas de prueba (rand() compartido, estado compartido, misma semilla, sin reduction, default(none) y la verificación del inicio de cada hilo), los scripts que los ejecutan en nuestro equipo con capturas de la terminal y un borrador de las respuestas	Aceptado: usar una prueba por cada defecto, porque así cada afirmación tiene una salida real que la respalda. Descartado: una primera medición de T(1) tomada con el portátil en batería; la repetimos conectados a la corriente. Revisamos todas las cifras contra las salidas de Parte_II/resultados/.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

El desempeño de un programa paralelo se mide con dos magnitudes. El speedup con p hilos es S(p) = T(1)/T(p), donde T(p) es el tiempo de ejecución con p hilos, y la eficiencia es E(p) = S(p)/p. Si una fracción f del trabajo es paralelizable y el resto es secuencial, la ley de Amdahl da el speedup máximo alcanzable:

$$S(p) = 1 / [\ (1 - f) + f/p\ ]$$

Protocolo experimental. Con $N = 10^8$ fijo, mida T(p) para $p \in \{1, 2, 4, 8\}$. Para cada valor de p realice al menos cinco ejecuciones y registre la mediana de los tiempos; justifique por qué la mediana es más apropiada que el promedio en este experimento.

Equipo. Las mediciones de esta parte se hicieron en el portátil de otro integrante del grupo, distinto del usado en la Parte II: un **Intel Core i5-1235U** de 12.ª generación con Windows y Ubuntu en WSL. Es un procesador híbrido con **10 núcleos**: 2 núcleos de rendimiento (P-cores), con hiperprocesamiento y hasta 4,4 GHz, y 8 núcleos de eficiencia (E-cores), sin hiperprocesamiento y hasta 3,3 GHz, para 12 hilos lógicos en total, con 12 MB de caché L3 compartida y un consumo base de 15 W. Dentro de WSL, `lscpu` lo reporta como 6 núcleos con 2 hilos cada uno (12 CPU), porque la máquina virtual no muestra la topología real; su salida completa se adjunta como anexo.

Procedimiento. Usamos el programa final de la Parte II (`montecarlo.c`, con 1000 bloques de 100 000 muestras y `schedule(static)`), compilado con `gcc -O2 -fopenmp montecarlo.c -o montecarlo -lm`, con $N = 10^8$ y la semilla 20261018. Con el portátil conectado a la corriente y después de una corrida de calentamiento, ejecutamos el programa cinco veces para cada $p \in \{1, 2, 4, 8\}$, eligiendo p con el segundo argumento (`./montecarlo 100000000 p`). El tiempo lo mide el propio programa con `omp_get_wtime()` alrededor del ciclo paralelo. En las 20 corridas el programa obtuvo los mismos 78 540 833 aciertos ($\pi = 3{,}141633$, error $4{,}07 \times 10^{-5}$), igual que en la Parte II, así que las diferencias de tiempo vienen solo de la ejecución.

¿Qué T(1) usamos? Como referencia tomamos la mediana con p = 1 medida en este mismo equipo, **T(1) = 0,6358 s**, y no el 1,3570 s de II.1. Ese valor se midió en otro procesador, que es unas 2,1 veces más lento en este cálculo. Al principio cometimos ese error: con T(1) = 1,3570 s el speedup con 2 hilos salía 4,19, mayor que p, y al despejar Amdahl obteníamos f = 1,149, una fracción imposible porque f no puede pasar de 1. Eso nos mostró que el speedup solo tiene sentido si T(1) y T(p) se miden en el mismo equipo. Usamos el programa paralelo con p = 1 porque es el mismo código que se ejecuta con más hilos, de modo que S(p) mide únicamente el efecto de agregar hilos.

¿Por qué la mediana y no el promedio?

Los tiempos de un portátil tienen ruido que no es simétrico. Una tarea de Windows en segundo plano, la capa de virtualización de WSL o el procesador bajando la frecuencia por temperatura pueden hacer que una corrida tarde bastante más, pero casi nunca que tarde mucho menos que el costo real del cálculo. Esos valores atípicos se van hacia un solo lado y arrastran el promedio. La mediana es el valor del medio una vez ordenados los tiempos, así que con cinco corridas puede haber hasta dos atípicas sin que cambie. Por eso representa mejor el tiempo típico del programa, y es la misma razón por la que la usamos para T(1) en II.1.

En nuestros datos se ve con p = 4: una corrida tardó 0,1966 s, por encima de las otras cuatro (entre 0,1830 y 0,1898 s). El promedio sube a 0,1885 s y la mediana se queda en 0,1880 s. Aquí la diferencia es pequeña porque las mediciones fueron estables (la mayor dispersión fue 0,019 s, con p = 1), pero con una sola corrida interrumpida por Windows el promedio se movería mucho más, y la mediana no.

Tabla III-1. Tiempos medidos con N = 10^8 en el Intel Core i5-1235U (cinco corridas por valor de p).

p (hilos)	Tiempos medidos [s]	Mediana de T(p) [s]	S(p) = T(1)/T(p)	E(p) = S(p)/p
1	0,6209 · 0,6352 · 0,6358 · 0,6365 · 0,6395	0,6358	1,000	1,000
2	0,3217 · 0,3235 · 0,3241 · 0,3283 · 0,3306	0,3241	1,962	0,981
4	0,1830 · 0,1852 · 0,1880 · 0,1898 · 0,1966	0,1880	3,382	0,845
8	0,1172 · 0,1181 · 0,1200 · 0,1206 · 0,1217	0,1200	5,298	0,662

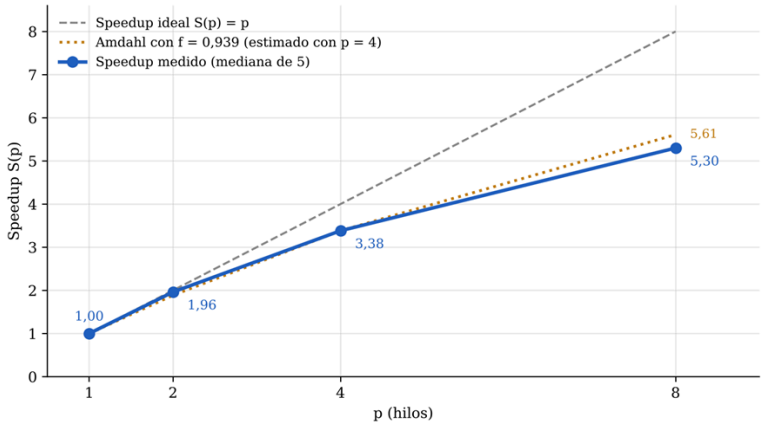


Figura III-1. Speedup medido frente al speedup ideal S(p) = p y frente a la ley de Amdahl con la fracción f estimada en III.2.

El programa escala bien pero no de forma ideal: con 2 hilos la eficiencia es 0,98, con 4 baja a 0,85 y con 8 a 0,66. Con 8 hilos el cálculo pasa de 0,6358 s a 0,1200 s, 5,3 veces más rápido.

III.1 Aplique la ley de Amdahl a los siguientes casos y interprete cada resultado: (a) f = 0,98, p = 8; (b) f = 0,90, p = 8; (c) f = 0,98 y p creciendo sin límite. ¿Qué le enseña el caso (c) sobre la fracción secuencial de un programa?

(a) f = 0,98, p = 8. S = 1 / (0,02 + 0,98/8) = 1 / (0,02 + 0,1225) = 1 / 0,1425 ≈ **7,02**, con eficiencia 7,02/8 ≈ 0,88. Aunque solo el 2 % del trabajo es secuencial, ya se pierde el 12 % de la capacidad de los 8 hilos. En el denominador ese 2 % pesa 0,02, mientras que el 98 % repartido pesa 0,1225: la parte secuencial ya es casi la sexta parte del tiempo total.

(b) f = 0,90, p = 8. S = 1 / (0,10 + 0,90/8) = 1 / (0,10 + 0,1125) = 1 / 0,2125 ≈ **4,71**, con eficiencia ≈ 0,59. Pasar de 2 % a 10 % de código secuencial reduce el speedup de 7,02 a 4,71. Con 8 hilos, la parte secuencial (0,10) ya cuesta casi lo mismo que toda la parte paralela (0,1125).

(c) f = 0,98 y p → ∞. El término f/p tiende a cero, así que S → 1 / (1 − f) = 1 / 0,02 = **50**. Por más núcleos que se agreguen, el programa nunca será más de 50 veces más rápido.

El caso (c) enseña que el techo lo pone la fracción secuencial y no el número de procesadores. Esa parte la ejecuta un solo hilo, así que el tiempo nunca baja de (1 − f)·T(1) por muchos núcleos que haya. Para que un programa escale, reducir lo secuencial vale más que agregar núcleos: bajarlo de 2 % a 1 % sube el techo de 50 a 100.

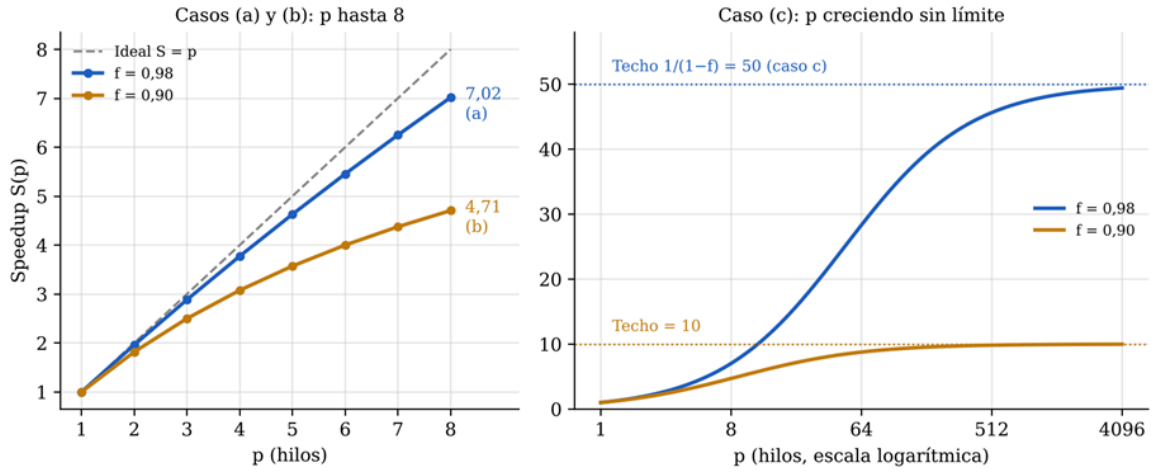


Figura III-2. Ley de Amdahl para f = 0,98 y f = 0,90. A la izquierda, los casos (a) y (b) con p hasta 8; a la derecha, el caso (c): al crecer p el speedup se acerca a 1/(1 − f) y nunca lo supera.

III.2 Con sus mediciones, estime la fracción secuencial f de su programa: despeje f de la ley de Amdahl a partir del speedup medido con p = 4 y muestre el procedimiento completo. Si lo desea, contraste su resultado con la métrica de Karp–Flatt, e = (1/S(p) − 1/p) / (1 − 1/p), y explique qué mide.

Despeje. Invertimos la ley de Amdahl y agrupamos los términos con f:

$$1/S = (1 - f) + f/p = 1 - f\cdot(1 - 1/p)$$

$$f = (1 - 1/S) / (1 - 1/p)$$

Cálculo con p = 4. Con las medianas de la Tabla III-1:

- S(4) = T(1) / T(4) = 0,6358 / 0,1880 = 3,382
- 1/S(4) = 0,2957, así que el numerador es 1 − 1/S = 0,7043
- Denominador: 1 − 1/4 = 0,75
- f = 0,7043 / 0,75 = **0,939**, y la fracción "secuencial" es 1 − f = **0,061** (6,1 %)

¿Qué tan razonable es ese valor? En el código de la Parte II, lo único que se ejecuta fuera del ciclo es leer los argumentos, verificar que `saltar` coincide con `rand_r`, calcular el número de bloques e imprimir el resultado. Todo eso toma microsegundos frente a 0,64 s de cómputo, menos del 0,01 %. Ese 6,1 % no corresponde a instrucciones secuenciales: es una fracción "efectiva" que mezcla costos que Amdahl no distingue, sobre todo que con más hilos activos los núcleos trabajan a menor frecuencia y que parte de los hilos corre en núcleos de eficiencia, más lentos. Lo analizamos en III.3.

Contraste con Karp–Flatt. La métrica e = (1/S − 1/p) / (1 − 1/p) estima la fracción secuencial *experimental*: incluye el código secuencial y también todo lo que impide escalar, como crear los hilos, sincronizarlos, el desbalance de carga o la pérdida de frecuencia. Con p = 4 da exactamente 1 − f, porque es el mismo despeje. Lo útil es calcularla para varios p:

p	S(p)	e (Karp–Flatt)	f = 1 − e
2	1,962	0,020	0,980
4	3,382	0,061	0,939
8	5,298	0,073	0,927

Si la pérdida viniera de una parte secuencial fija del código, e sería más o menos constante. En cambio, e se triplica de p = 2 a p = 4 (de 0,020 a 0,061) y sigue subiendo con p = 8 (0,073). Eso indica que la pérdida es un sobrecosto que crece al agregar hilos, no código secuencial. El salto grande entre 2 y 4 hilos coincide con lo que esperaríamos en este procesador: con 2 hilos los dos pueden correr en los núcleos de rendimiento, mientras que con 4 al menos dos tienen que usar núcleos de eficiencia o compartir un núcleo de rendimiento. Por eso el f que da Amdahl depende del p con que se calcule; en lo que sigue usamos el que pide el enunciado, **f = 0,939**.

III.3 Compare su speedup medido con el ideal lineal S(p) = p y con el predicho por la ley de Amdahl usando su f estimado. Explique las discrepancias que observe. Considere en su análisis, entre otros factores, la creación y destrucción de hilos, el

costo de la reducción, los límites de ancho de banda de memoria y la jerarquía de caché, y el efecto de la frecuencia variable del procesador y del hiperprocesamiento.

p	Ideal S = p	Amdahl (f = 0,939)	Medido	Medido / Amdahl
2	2	1,885	1,962	1,04
4	4	3,382	3,382	1,00
8	8	5,608	5,298	0,94

El speedup medido siempre queda por debajo del ideal, y la distancia crece con p: con 8 hilos se obtiene 5,30 en lugar de 8. Frente a Amdahl ocurre algo distinto según p: con 2 hilos el programa escala **mejor** que lo predicho (1,96 frente a 1,89) y con 8 hilos **peor** (5,30 frente a 5,61). Esto confirma lo que vimos con Karp–Flatt: Amdahl supone una fracción secuencial fija, pero en nuestro programa la pérdida es pequeña con pocos hilos y crece con muchos. Analizamos cada factor:

- **Creación y destrucción de hilos.** El programa abre una sola región paralela, y crear el equipo de hilos cuesta del orden de microsegundos frente a más de 0,1 s de cómputo. Calcular los saltos del generador tampoco pesa: son 1000 saltos de unas cien multiplicaciones cada uno, frente a 2×10^8 llamadas a `rand_r`. Este factor no explica las diferencias.
- **Costo de la reducción.** Cada hilo cuenta en su copia privada de `dentro` y OpenMP suma las p copias una sola vez al final: son como máximo 8 sumas. Es despreciable, a diferencia de lo que vimos en II.4 sin la cláusula, donde la variable compartida hacía perder aciertos.
- **Ancho de banda de memoria y jerarquía de caché.** El ciclo no recorre arreglos: el estado del generador, x, y y el contador caben en registros. El programa está limitado por cálculo y no por memoria, así que ni el ancho de banda ni la caché L3 compartida de 12 MB son un cuello de botella. Tampoco hay *false sharing*, porque cada hilo escribe solo en su copia privada.
- **Núcleos de distinta velocidad (arquitectura híbrida).** Es la causa más particular de este equipo. Los núcleos de eficiencia llegan a 3,3 GHz frente a 4,4 GHz de los de rendimiento. Con `schedule(static)` cada hilo recibe la misma cantidad de bloques (500, 250 o 125 para p = 2, 4 y 8); si alguno cae en un núcleo lento, termina después, y los demás deben esperarlo antes de la reducción. Esto es un desbalance de carga causado por el hardware y no por el programa. No controlamos en qué núcleo corre cada hilo, porque WSL oculta la topología real, pero explica bien que la eficiencia caiga de 0,98 a 0,85 al pasar de 2 a 4 hilos. Una planificación `dynamic` con bloques grandes podría compensarlo, porque los núcleos rápidos tomarían más bloques.

- **Frecuencia variable (turbo).** Con un solo hilo activo, el procesador puede llevar ese núcleo cerca de su máximo; con muchos núcleos activos, el límite de consumo de un portátil de 15 W obliga a bajar la frecuencia de todos. Por eso T(1) se midió con el núcleo más rápido posible, y con 8 hilos cada núcleo hace menos trabajo por segundo. Esto reduce el speedup aunque el programa sea casi totalmente paralelo.
- **Hiperprocesamiento.** En este procesador solo los 2 núcleos de rendimiento tienen dos hilos lógicos. Si dos hilos nuestros comparten uno de esos núcleos, compiten por las mismas unidades de cálculo, y en un ciclo tan aritmético como este cada uno avanza más lento que en un núcleo propio. Con p = 8 todavía hay más núcleos físicos (10) que hilos, pero el sistema no siempre los reparte de la mejor manera, y esto contribuye a que E(8) baje a 0,66.
- **WSL y el sistema operativo.** Los hilos corren dentro de la máquina virtual de WSL, que comparte el procesador con Windows. Cualquier tarea de Windows le quita tiempo a alguno de nuestros hilos y retrasa a los demás, que deben esperar al más lento. Con 8 hilos hay menos núcleos libres para esas tareas, así que el efecto es mayor.

III.4 Suponga que le ofrecen tiempo de cómputo en una máquina con 64 núcleos. Con su fracción f estimada, ¿qué speedup máximo podría obtener y en qué se diferenciaría de crecer el número de muestras N? Reflexione: aumentar p con N fijo, ¿mejora la precisión del resultado o solo el tiempo de cómputo? ¿Qué mejora cada una de las dos variables?

Speedup con 64 núcleos. Con f = 0,939 de III.2:

$$S(64) = 1 / (0{,}061 + 0{,}939/64) = 1 / (0{,}061 + 0{,}0147) \approx 13{,}2$$

El tiempo pasaría de 0,6358 s a unos 0,048 s, pero con una eficiencia de apenas 13,2/64 ≈ 0,21, y el techo para cualquier número de núcleos sería 1/(1 − f) ≈ 16,4. Hay que leer este número con cuidado. El f de III.2 incluye efectos propios de nuestro portátil (núcleos de distinta velocidad, pérdida de turbo, WSL) que un servidor de 64 núcleos iguales, con frecuencia estable, no tendría. Allí f sería mucho más cercano a 1: con f = 0,98 de la estimación con p = 2, el speedup sería ≈ 28, y con el f del código (≈ 0,999) ≈ 60. También habría un límite propio de nuestro programa: con 1000 bloques y 64 hilos, unos hilos reciben 16 bloques y otros 15, así que todos esperan a los de 16 y la eficiencia no puede pasar de 15,625/16 ≈ 0,98. Con más núcleos convendría usar bloques más pequeños.

Aumentar p con N fijo no mejora la precisión. En las 20 corridas de la Tabla III-1 el programa dio exactamente los mismos 78 540 833 aciertos con p = 1, 2, 4 y 8: más hilos procesan los mismos puntos, solo que en menos tiempo. Como vimos en la Parte I (I.1 e I.3) y en II.1, el error depende solo del número total de muestras:

$$\sigma = 4 \cdot \sqrt{[\ (\pi/4)(1 - \pi/4) / N\ ]} \approx 1{,}64 / \sqrt{N}$$

Con $N = 10^8$ el error típico es $\approx 1{,}64 \times 10^{-4}$, y nuestro error real es $4{,}07 \times 10^{-5}$, sea cual sea p. Con 64 núcleos y el mismo N tendríamos exactamente el mismo π, solo que más rápido.

Qué mejora cada variable. Aumentar **p** reduce el **tiempo**. Aumentar **N** reduce el **error**, pero lentamente: según I.1, reducir el error a la mitad exige 4N muestras y ganar un decimal exige 100N. La forma de aprovechar los 64 núcleos para ganar precisión es combinar ambas cosas: usar 64 veces más muestras ($N = 6{,}4 \times 10^9$) en un tiempo parecido al actual. Eso divide el error entre $\sqrt{64} = 8$, hasta $\approx 2 \times 10^{-5}$: menos de un decimal adicional a cambio de 64 veces más hardware.

Para ese N nuestro programa necesitaría un cambio. Como dijimos en II.2, el generador de `rand_r` repite su secuencia después de 2^{32} pasos y cada muestra usa 6, así que el programa solo es válido hasta $N \approx 7{,}2 \times 10^8$ (por eso imprime un aviso si N lo supera). Habría que usar un generador de 64 bits, al que se le puede aplicar la misma técnica de salto.

III.5 (Opcional.) Discuta brevemente por qué la ley de Amdahl puede resultar pesimista cuando, en lugar de fijar el tamaño del problema, lo que se fija es el tiempo total de cómputo (argumento de Gustafson).

Amdahl supone que N está fijo y pregunta cuánto baja el tiempo al agregar núcleos. Pero cuando hay más núcleos normalmente no se resuelve el mismo problema más rápido, sino uno más grande en el mismo tiempo: es lo que planteamos en III.4 al pasar a $N = 6{,}4 \times 10^9$. Si s es la fracción secuencial medida en la ejecución paralela, el speedup escalado de Gustafson es:

$$S_G(p) = s + (1 - s) \cdot p = p - s \cdot (p - 1)$$

Al crecer N, la parte paralela (el ciclo de muestreo) crece en proporción, pero la parte secuencial (leer argumentos, verificar el generador, imprimir) sigue igual. Así s se hace cada vez más pequeña y deja de imponer un techo fijo. Incluso con nuestra $s = 1 - f = 0{,}061$ y $p = 64$, Gustafson da $S_G = 64 - 0{,}061 \cdot 63 \approx 60{,}2$, frente a 13,2 de Amdahl.

En Monte Carlo esta visión es la más natural, porque el motivo para usar más núcleos es justamente procesar más muestras y reducir el error, como vimos en la Parte I. Por eso Amdahl resulta pesimista aquí: mide cuánto se acelera un N fijo, que no es lo que uno haría con una máquina más grande.

[C1]Ojo: estos tiempos y el lscpu salieron de una máquina virtual de 2 núcleos donde Claude corrió el código, no de un equipo del grupo. Antes de entregar hay que repetir las mediciones (con la semilla asignada) en el computador de uno de nosotros, actualizar la Tabla 1, la Figura 2, los cálculos de III.2–III.4 y adjuntar ese lscpu. Registrar en la bitácora de IA.